@holokai/neuron-sdk

SDK for building neurons — processes that connect to the BigBrain gateway and execute tasks on behalf of workflows.

A neuron advertises one or more **capabilities** (typed task handlers), opens a persistent connection to the gateway, and receives work via a pluggable `Transport`. Two transports ship in the SDK:

- **`HttpTransport`** — SSE for inbound, HTTPS POST for outbound. The default for any neuron talking to a remote BigBrain gateway. Re-exported from the package root.
- **`IpcTransport`** — Electron renderer ↔ main IPC. For neurons running inside an app that embeds BigBrain in main; no network hop, no JWT in the renderer. Imported from the `/ipc` subpath.

The gateway routes tasks to the right neuron based on capability type and scope. The built-in `apps/worker` uses this SDK over HTTP; the in-process Holokai Desktop renderer uses it over IPC; both share the same `Neuron` class and handler API.

Installation

```
pnpm add @holokai/neuron-sdk
# or: npm install @holokai/neuron-sdk
# or: yarn add @holokai/neuron-sdk
```

Runs in **Node 20+** (uses the global `fetch` and `ReadableStream`) and **modern browsers** (Chromium, Firefox, Safari latest). The package has no `node:*` imports — Holokai Desktop / Electron and the in-browser neuron-tester both consume the same SDK build.

> **Gateway compatibility (protocol 1.1.0, HOL-2926).** Since protocol 1.1.0 the SDK advertises each capability's contract (`description` + input/output JSON Schemas) on the register frame. The gateway validates register frames **strictly**, so a gateway older than HOL-2926 rejects them with `INVALID_FRAME` — **deploy the gateway before publishing or rolling out any 1.1.0-protocol SDK (including neuron-tester OTA)**. A rejected registration leaves the SSE stream open but the neuron lease-less; the SDK surfaces it via the `error` event (`registration rejected by gateway: ...`) so apps can react.

Quick start

```ts
import { Neuron } from '@holokai/neuron-sdk';

const neuron = new Neuron({
  gatewayUrl: 'https://bigbrain.holokai.dev',
  neuronId: 'my-worker-001',            // stable per process/device
  auth: () => process.env.GATEWAY_TOKEN!, // or an async refresh function
});

neuron.handle(
  { type: 'core/script.python', scope: 'any', concurrency: 4 },
  async (input, ctx) => {
    ctx.log.info({}, 'running python script');
    const result = await runPython(input as { code: string }, ctx.signal);
    return { output: result };
  },
);

await neuron.start();

process.on('SIGTERM', () => neuron.stop({ drain: true, timeoutMs: 30_000 }));
```

Authentication

The `auth` callback supplies the bearer token for every POST and SSE open. How you obtain tokens depends on where the neuron runs (full model: `docs/NEURON_AUTH_SPEC.md` in the bigbrain repo):

- **Desktop-embedded neurons** run under the signed-in user: exchange the user's Moku JWT for an `aud=bigbrain` token (RFC 8693) and return it from `auth`.
- **Headless / enterprise neurons** enroll once via Moku's device flow, then mint short-lived tokens from the stored client credential — no static long-lived JWT anywhere.

Enroll (one-time, headless neurons)

```sh
npx @holokai/neuron-sdk enroll --moku-url https://moku.example \
  --name warehouse-connector --capabilities org/acme/erp.read,org/acme/erp.write
```

Prints a verification URL + user code; an org admin approves in Moku. The client credential is written to `~/.holokai/neuron-credentials.json` (`0600`; override with `--credentials-file`). The requested capabilities are display hints for the approval screen — what the neuron may actually advertise is governed by the org's capability catalog. Polling is resilient: transient server errors (409/5xx) and network blips are retried until the device code expires; only a denial, expiry, or protocol error is terminal.

Also available programmatically: `enroll(opts)` from `@holokai/neuron-sdk/auth`.

Steady state

```ts
import { Neuron } from '@holokai/neuron-sdk';
import { createClientCredentialsAuth, defaultCredentialPath } from '@holokai/neuron-sdk/auth'

const neuron = new Neuron({
  gatewayUrl: 'https://bigbrain.holokai.dev',
  neuronId: 'warehouse-connector-001',
  auth: createClientCredentialsAuth(defaultCredentialPath()),
});
```

`createClientCredentialsAuth` mints `aud=bigbrain` tokens via the OAuth2 `client_credentials` grant, caches them against the response's `expires_in` (proactive refresh, single-flight), and re-reads the credential file on `invalid_client` so an operator-rotated secret is picked up without a restart. If the credential was revoked in Moku it throws `CredentialRevokedError` — re-enroll to recover.

> `@holokai/neuron-sdk/auth` is **Node-only** (it reads the credential file); the root and `./ipc` entries remain browser-clean. Python SDK parity ships in `holokai-neuron-sdk` ≥ 0.2.0 (`holokai_neuron_sdk.auth`: `holokai-neuron-sdk enroll` + `create_client_credentials_auth`) — same credential file, interchangeable across both SDKs.

Core concepts

Capability

A capability is a named contract a neuron can fulfill. It has:

Field	Type	Description
`type`	`string`	Namespaced identifier: `{owner}/{package}[/{name}][.{variant}]`
`scope`	`'any' | { onBehalfOf: userId }`	Who this handler serves
`concurrency`	`number`	Max parallel tasks for this capability

Examples:

- `core/script.python` — server-side Python execution, any user
- `desktop/mcp/drive.read` — user's Google Drive, scoped to that user
- `org/01HBQ8ZK/custom-tool` — org-specific capability

Namespace authorization

The gateway enforces which identities may advertise which namespaces:

Namespace prefix	Permitted advertisers	Required scope
`core/*`	Server service identities	`any`
`desktop/*`	Attested desktop binaries	`{ onBehalfOf: userId }`
`org/{orgId}/*`	Tenant-installed extensions	`any` within org
`user/{userId}/*`	Personal extensions	`{ onBehalfOf: userId }`

Registration is rejected (HTTP 403) if the JWT is not authorized for the namespace.

Scope

- **`'any'`** — the neuron handles tasks for any user. Shared queue; no per-user binding.
- **`{ onBehalfOf: userId }`** — the neuron handles tasks for that specific user only. The JWT's user binding must match; the gateway enforces this at registration time.

One neuron may advertise mixed-scope capabilities:

```
neuron.handle({ type: 'desktop/mcp/drive.read', scope: { onBehalfOf: 'user:abc' }, concurrenc
neuron.handle({ type: 'core/script.python',     scope: 'any',                      concurren
```

Lease

A lease is how the gateway delivers a task. When the broker dispatches a task to a neuron, the gateway:

1. Writes a DB row with `leaseId` and expiry.
2. Sends a `lease` frame over SSE.

The SDK validates input, invokes your handler, and sends either `ack` (success) or `nack` (failure) over HTTPS POST. Until the ack arrives, the message remains unacknowledged in the broker — if the neuron disconnects, the broker redelivers automatically.

Handlers must be idempotent. A task that completed a side effect but failed to ack will be retried. Use capability-native idempotency keys (e.g., Google Drive `requestId`, HTTP `Idempotency-Key`).

Neuron ID

`neuronId` must be **stable across restarts** for the same process/device. The gateway uses it to correlate reconnects. Pick something meaningful and persistent:

```
// Server process
neuronId: `worker-${process.env.HOSTNAME}-${process.pid}`

// Desktop app — persist a UUID to disk
neuronId: await readOrCreateStoredId('/var/lib/myapp/neuron-id')
```

API reference

`new Neuron(opts: NeuronOptions)`

`NeuronOptions` is a discriminated union — pick the convenience shape (HTTP) or pass a transport explicitly.

```
type NeuronOptions = NeuronOptionsWithHttp | NeuronOptionsWithTransport;

interface NeuronCommonOptions {
  neuronId: string;             // Stable per process/device
  logger?: NeuronLogger;        // Plug in pino, console, etc. Default: silent
  heartbeatIntervalMs?: number; // Default 10_000 (10s)
}

// Convenience: SDK builds an HttpTransport for you. Most common path.
interface NeuronOptionsWithHttp extends NeuronCommonOptions {
  gatewayUrl: string;           // Base URL, e.g. 'https://bigbrain.holokai.dev'
  auth: AuthCallback;           // Token provider (see below)
  initialReconnectDelayMs?: number; // Default 500ms
  maxReconnectDelayMs?: number;     // Default 30_000 (30s)
  fetch?: typeof fetch;         // Override for tests
}

// Power-user: bring your own Transport. Used for IpcTransport, custom
// transports, or HttpTransport instances built explicitly.
interface NeuronOptionsWithTransport extends NeuronCommonOptions {
  transport: Transport;
}
```

See the Transports section for `HttpTransport`, `IpcTransport`, and the `Transport` interface.

`neuron.handle(reg, handler)`

Register a handler for a capability. Must be called **before** `neuron.start()`.

```
neuron.handle(reg: CapabilityRegistration, handler: Handler): void
```

```ts
interface CapabilityRegistration {
  type: string;                  // Namespaced capability type
  scope: 'any' | { onBehalfOf: string };
  concurrency: number;           // Max parallel executions
  inputSchema?: JsonSchema;      // Optional; validated before invoking handler
  outputSchema?: JsonSchema;     // Optional; validated before acking
}

type Handler<TInput = unknown, TOutput = unknown> = (
  input: TInput,
  ctx: HandlerContext,
) => Promise<TOutput> | TOutput;
```

If `inputSchema` is provided and the lease input fails validation, the handler is never called — the SDK sends a `schema-mismatch` nack automatically. Same for `outputSchema` on the return value.

Schema precedence. For each of input and output the SDK picks exactly one validator:

1. The zod schema from `defineCapability`, when the capability was registered that way. JSON Schema can't express `.refine()`, `.transform()`, or `.preprocess()`, so the zod contract stays authoritative and its transforms are what the handler sees and what goes on the wire.
2. Otherwise the JSON Schema passed here on `CapabilityRegistration`.

A capability registered with neither is not validated client-side — its handler runs on whatever arrives.

The `inputSchema` / `outputSchema` that ride on the lease frame are **not** consulted. They can only ever be your own schemas handed back to you — the register frame carries no schemas, so the server's only source is the `POST /plan` handler catalog the client supplies. And the planner doesn't persist those onto task rows, so on real workflow tasks the lease frame's schemas are absent entirely; the only case where they arrive populated is the dev-only `/__dev__/inject-task` route used by neuron-tester, which echoes back what the capability registered.

> **This is the only validation that runs.** The gateway's ack-time check validates against that same (empty) `lease.outputSchema`, so it does not currently catch anything. If a capability declares no schema here, its input and output are unchecked end to end.

> **Note:** the Go and Python SDKs currently prefer the lease frame's schema and fall back to the registered one. TypeScript deliberately diverges here; the other two are expected to follow.

`neuron.start()`

Opens the SSE stream, posts `register` with all advertised capabilities, and starts the heartbeat loop.

```ts
await neuron.start(): Promise<void>
```

Throws if no handlers have been registered. Subsequent calls are no-ops.

`neuron.stop(opts?)`

Graceful shutdown: refuse new leases, drain in-flight handlers, disconnect.

```
await neuron.stop({ drain?: boolean, timeoutMs?: number }): Promise<void>
```

Option	Default	Description
`drain`	`true`	Wait for in-flight handlers to finish
`timeoutMs`	`30_000`	After this, abort remaining in-flight tasks

Queued leases (received but not yet started due to concurrency limits) are nacked `retryable` on shutdown so the broker redelivers immediately.

`neuron.updateCapability(reg)`

Change concurrency for an already-registered capability at runtime. Useful when available resources change (e.g., desktop plugged into power).

```
await neuron.updateCapability({ type: string, concurrency: number }): Promise<void>
```

Notifies the gateway via `update-capability` so it can tune AMQP prefetch. Safe to call while tasks are in flight — in-flight tasks finish at the old concurrency; new slots open at the new limit.

`HandlerContext`

The second argument to every handler:

```
interface HandlerContext {
  task: TaskPayload;    // Full task metadata
  leaseId: string;      // Gateway lease ID (for logging/debugging)
  signal: AbortSignal;  // Fires on gateway cancel or SDK shutdown
  log: NeuronLogger;    // Logger scoped to this lease
  progress(update: { percent?: number; message?: string; data?: unknown }): void;
  fail(reason: string, opts?: { code?: string }): never;
}
```

`ctx.signal`

Pass this to any fetch/HTTP calls so they abort cleanly:

```ts
const resp = await fetch(url, { signal: ctx.signal });
```

When the gateway sends a `cancel` frame (workflow was cancelled by the user), `ctx.signal` fires and your handler should throw/return promptly. An `AbortError` from a cancelled signal is treated as `nack(kind='cancelled')` — benign, not counted as a failure.

`ctx.progress()`

Report progress for long-running tasks. Non-blocking; safe to call many times.

```ts
ctx.progress({ percent: 50, message: 'halfway done' });
ctx.progress({ data: { rowsProcessed: 1200 } });
```

`ctx.fail()`

Terminal failure — nacks the lease with `kind='terminal'` (no retry). Use when the task can never succeed regardless of retries.

```ts
if (!validApiKey) {
  ctx.fail('invalid API key', { code: 'INVALID_API_KEY' });
  // unreachable — fail() throws internally
}
```

`AuthCallback`

```ts
type AuthCallback = () => Promise<string> | string;
```

Called on every POST and SSE open. The SDK retries once on 401, so implement token refresh inside the callback:

```ts
let token = await fetchInitialToken();

const auth = async () => {
  if (isExpired(token)) {
    token = await refreshToken();
  }
  return token.accessToken;
};
```

`NeuronLogger`

```ts
interface NeuronLogger {
  debug(meta: Record<string, unknown>, msg?: string): void;
  info(meta: Record<string, unknown>, msg?: string): void;
  warn(meta: Record<string, unknown>, msg?: string): void;
  error(meta: Record<string, unknown>, msg?: string): void;
}
```

Matches pino's call signature directly. To use pino:

```ts
import pino from 'pino';
const logger = pino({ level: 'info' });
const neuron = new Neuron({ ..., logger });
```

`defineCapability()`

A typed helper that produces a capability definition from zod schemas, with handler inputs/outputs fully inferred:

```ts
import { defineCapability, Neuron } from '@holokai/neuron-sdk';
import { z } from 'zod';

const pythonCap = defineCapability({
  type: 'core/script.python',
  description: 'Run a Python script in a sandbox',
  input: z.object({ code: z.string(), args: z.record(z.unknown()).optional() }),
  output: z.object({ stdout: z.string(), exitCode: z.number() }),
  scope: 'any',          // optional default; overridable at handle() time
  concurrency: 4,        // optional default; overridable at handle() time
});

// Handler input/output inferred — no manual type annotations:
neuron.handle(pythonCap, async (input, ctx) => {
  // input: { code: string; args?: Record<string, unknown> }
  return { stdout: '...', exitCode: 0 };
});

// Or override defaults per deployment:
neuron.handle(pythonCap, { scope: { onBehalfOf: 'user-123' }, concurrency: 8 }, fn);
```

Validation uses the **original zod schemas** (so `.refine()`, `.transform()`, `.preprocess()`, `.brand()` all run as expected). The JSON Schema produced by `zod-to-json-schema` is shipped on the wire to the gateway and planner but is not used as the runtime validator — that avoids silently dropping zod features that don't translate.

The fall-back `neuron.handle(registration, fn)` shape (raw JSON Schema + capability fields) is still accepted for low-level callers that don't have zod schemas on hand.

Events

`Neuron` extends `EventEmitter` from `eventemitter3` — a browser-clean, type-safe drop-in for Node's `EventEmitter`. Subscribe with `.on()`, `.once()`, or `.off()`:

```
const off = neuron.on('frame', (frame) => {
  log.append(frame); // see every parsed SSE frame
});
off(); // unsubscribe

neuron.once('connection:registered', ({ sessionId }) => {
  console.log(`registered as ${sessionId}`);
});

neuron.on('connection:reconnecting', ({ attempt, delayMs, reason }) => {
  statusBar.show(`reconnecting (attempt ${attempt}, ${delayMs}ms)`, reason);
});
```

The full event map:

Event	Payload	When
`connection:open`	*none*	SSE handshake succeeded; before `register` is posted
`connection:registered`	`{ sessionId: string; capabilities: number }`	Gateway accepted the register frame
`connection:close`	`reason: string`	SSE stream ended (clean shutdown or transient error)
`connection:reconnecting`	`{ attempt, delayMs, reason }`	Reconnect scheduled, before backoff delay
`error`	`err: Error`	Transport / parse error (reconnect handling is automatic)
`frame`	`frame: ServerToNeuronFrame`	Pass-through of every parsed inbound SSE frame

The `frame` event is a firehose mirror of the gateway's `/neuron/events` SSE stream. It fires for **every** frame including ones the SDK handles internally (`lease`, `cancel`) — useful for debug UIs, observability, and audit logs. The high-level `neuron.handle()` API is implemented as a `frame` listener filtered to `lease` frames, so `handle()` and direct `frame` subscribers compose without conflict.

Subscribe **before** calling `neuron.start()` if you care about events that fire as part of the connection sequence (`connection:open`, the initial `registered` frame). Late subscribers will still see subsequent events.

The event surface is designed to grow — future entries (e.g. `'workflow:task-completed'` for cross-neuron notifications) slot in without breaking subscribers.

Transports

A `Transport` is the wire-level abstraction the rest of the SDK builds on. It carries typed frames in both directions: inbound `ServerToNeuronFrame`s via callbacks, outbound `ClientFrame`s via `post(...)`. The `Neuron` class only ever talks to a `Transport` — adding a new transport (e.g. WebSocket, native messaging) means implementing one interface.

```ts
interface Transport {
  subscribe(callbacks: TransportCallbacks): void;
  start(): Promise<void>;
  stop(): Promise<void>;
  post<T extends ClientFrame>(path: PostPath, frame: T): Promise<unknown>;
  isConnected(): boolean;
}

interface TransportCallbacks {
  onOpen?: () => void;
  onClose?: (reason: string) => void;
  onFrame?: (frame: ServerToNeuronFrame) => void;
  onError?: (err: Error) => void;
  onReconnecting?: (info: { attempt: number; delayMs: number; reason: string }) => void;
}
```

Both `Transport` and `TransportCallbacks` are exported from the package root.

`HttpTransport`

Default transport. SSE for inbound, HTTPS POST for outbound. Reconnects with exponential backoff (default: 500ms → 30s ceiling). Refreshes the auth token via your `AuthCallback` and retries once on 401.

```ts
import { Neuron, HttpTransport } from '@holokai/neuron-sdk';

const transport = new HttpTransport({
  gatewayUrl: 'https://bigbrain.holokai.dev',
  neuronId: 'my-worker-001',
  auth: () => process.env.GATEWAY_TOKEN!,
  // optional:
  initialReconnectDelayMs: 500,
  maxReconnectDelayMs: 30_000,
});

const neuron = new Neuron({ neuronId: 'my-worker-001', transport });
```

In practice you rarely construct `HttpTransport` directly — passing `gatewayUrl` + `auth` to `Neuron` builds it for you. Construct explicitly when you want to share a fetch implementation, swap the auth strategy, or instrument the transport in tests.

```ts
interface HttpTransportOptions {
  gatewayUrl: string;
  neuronId: string;
  auth: AuthCallback;
  logger?: NeuronLogger;
  initialReconnectDelayMs?: number; // default 500ms
  maxReconnectDelayMs?: number;     // default 30_000ms
  fetch?: typeof fetch;
}
```

`IpcTransport`

For renderer-process neurons running inside an Electron app that hosts BigBrain in the main process. Leases arrive over Electron IPC, not over the network. Imported from the `/ipc` subpath so non-Electron consumers don't pay for it:

```ts
import { Neuron } from '@holokai/neuron-sdk';
import { IpcTransport } from '@holokai/neuron-sdk/ipc';

// `bridge` comes from the renderer's preload — see below.
const transport = new IpcTransport({ bridge: window.bigbrainNeuron, neuronId });
const neuron = new Neuron({ neuronId, transport });

neuron.handle({ type: 'desktop/local-tool', scope: { onBehalfOf: userId }, concurrency: 1 },
await neuron.start();
```

Differences from `HttpTransport`:

- **No reconnect logic** — IPC has no transient socket. If the bridge throws, the failure surfaces via `onError` and the consumer decides what to do.
- **No auth callback** — the renderer never sees the JWT. The Electron main process attaches `Authorization` headers when it forwards POSTs to the embedded Fastify gateway.
- **No keep-alive** — IPC channels are process-local; nothing to time out.

The SDK does **not** depend on Electron. The renderer's preload script is responsible for constructing a concrete bridge that satisfies the `NeuronIpcBridge` shape and exposing it via `contextBridge`:

```ts
interface NeuronIpcBridge {
  attach(neuronId: string): Promise<{ channel: string }>;
  detach(neuronId: string): Promise<void>;
  subscribe(channel: string, cb: (frame: ServerToNeuronFrame) => void): () => void;
  invoke<T extends ClientFrame>(path: PostPath, frame: T): Promise<unknown>;
}
```

Typical preload wiring (Electron-side, app code — not part of the SDK):

```ts
// preload.ts
import { contextBridge, ipcRenderer } from 'electron';

contextBridge.exposeInMainWorld('bigbrainNeuron', {
  attach: (neuronId) => ipcRenderer.invoke('bigbrain:attach', neuronId),
  detach: (neuronId) => ipcRenderer.invoke('bigbrain:detach', neuronId),
  subscribe: (channel, cb) => {
    const listener = (_e, frame) => cb(frame);
    ipcRenderer.on(channel, listener);
    return () => ipcRenderer.off(channel, listener);
  },
  invoke: (path, frame) => ipcRenderer.invoke('bigbrain:post', path, frame),
});
```

```ts
interface IpcTransportOptions {
  bridge: NeuronIpcBridge;
  neuronId: string;
  logger?: NeuronLogger;
}
```

Examples

Simple any-scoped capability

```ts
import { Neuron } from '@holokai/neuron-sdk';

const neuron = new Neuron({
  gatewayUrl: process.env.GATEWAY_URL!,
  neuronId: `worker-${process.env.HOSTNAME}`,
  auth: () => process.env.GATEWAY_TOKEN!,
});

neuron.handle(
  { type: 'core/http', scope: 'any', concurrency: 8 },
  async (input, ctx) => {
    const { url, method = 'GET', headers = {}, body } = input as {
      url: string;
      method?: string;
      headers?: Record<string, string>;
      body?: string;
    };

    const resp = await fetch(url, { method, headers, body, signal: ctx.signal });
    const text = await resp.text();

    return { status: resp.status, body: text };
  },
);

await neuron.start();
```

User-scoped capability (desktop / MCP)

```ts
const userId = await getUserIdFromToken(authToken);

neuron.handle(
  { type: 'desktop/mcp/drive.read', scope: { onBehalfOf: userId }, concurrency: 1 },
  async (input, ctx) => {
    const { fileId } = input as { fileId: string };
    ctx.progress({ message: 'fetching file metadata' });

    const file = await driveClient.files.get({ fileId, signal: ctx.signal });
    ctx.progress({ percent: 50, message: 'downloading content' });

    const content = await driveClient.files.download({ fileId, signal: ctx.signal });
    return { name: file.name, mimeType: file.mimeType, content };
  },
);
```

Long-running task with progress

```
neuron.handle(
  { type: 'core/data.transform', scope: 'any', concurrency: 2 },
  async (input, ctx) => {
    const { rows } = input as { rows: unknown[] };
    const results: unknown[] = [];

    for (let i = 0; i < rows.length; i++) {
      if (ctx.signal.aborted) break; // respect cancellation

      results.push(await processRow(rows[i]));
      ctx.progress({ percent: Math.round(((i + 1) / rows.length) * 100) });
    }

    return { results };
  },
);
```

Error handling

```
neuron.handle(
  { type: 'core/llm', scope: 'any', concurrency: 4 },
  async (input, ctx) => {
    const { prompt, model } = input as { prompt: string; model: string };

    let response;
    try {
      response = await llmClient.complete({ prompt, model, signal: ctx.signal });
    } catch (err) {
      if ((err as Error).name === 'AbortError') {
        throw err; // let the SDK handle cancellation
      }
      if ((err as any).status === 400) {
        // Bad request — no point retrying
        ctx.fail(`invalid LLM request: ${(err as Error).message}`, { code: 'INVALID_REQUEST'
      }
      // Anything else: throw to get a retryable nack
      throw err;
    }

    return { text: response.text, tokensUsed: response.usage.total };
  },
);
```

Graceful shutdown

```ts
await neuron.start();

const shutdown = async (signal: string) => {
  console.log(`${signal} received — draining...`);
  await neuron.stop({ drain: true, timeoutMs: 30_000 });
  process.exit(0);
};

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT',  () => shutdown('SIGINT'));
```

Runtime concurrency update

```ts
import { powerMonitor } from 'electron'; // or any power/resource API

powerMonitor.on('on-ac', async () => {
  await neuron.updateCapability({ type: 'core/script.python', concurrency: 8 });
});

powerMonitor.on('on-battery', async () => {
  await neuron.updateCapability({ type: 'core/script.python', concurrency: 1 });
});
```

Configuration reference

Option	Default	Applies to	Notes
`neuronId`	—	both	Stable per device/process; reused on reconnect
`logger`	silent no-op	both	Plug in pino or console
`heartbeatIntervalMs`	`10_000`	both	Sent every 10s; renews lease expiries
`transport`	—	with-transport shape	Pre-built `Transport` (e.g. `IpcTransport` or a custom `HttpTransport`)
`gatewayUrl`	—	with-http shape	Base URL, no trailing slash
`auth`	—	with-http shape	Called on every POST + SSE open
`initialReconnectDelayMs`	`500`	with-http shape	First reconnect delay (HTTP only — IPC doesn't reconnect)
`maxReconnectDelayMs`	`30_000`	with-http shape	Reconnect delay ceiling (exponential backoff)
`fetch`	global `fetch`	with-http shape	Override for tests

Protocol details

This section covers enough to debug; the wire-level frame schemas are bundled into the SDK and exported alongside the public types.

Transport

The SDK abstracts the wire as a `Transport` (see Transports above). The frame types below are the same regardless of transport — only the carrier differs.

`HttpTransport` (default):

- **Server → Neuron**: SSE stream at `GET /neuron/events?neuronId={id}`
- **Neuron → Server**: HTTPS POST endpoints (one per frame type)
- No WebSockets — SSE + HTTPS POST traverses enterprise proxies cleanly.

`IpcTransport` (Electron renderer):

- **Server → Neuron**: per-neuron Electron IPC channel obtained from `bridge.attach()`.
- **Neuron → Server**: `bridge.invoke(path, frame)`, which the main process forwards to the embedded Fastify gateway via `app.inject(...)` — same auth and validation chain as HTTP.
- No reconnect, no auth callback, no keep-alive.

Frame types

Neuron → Server (POST):

Frame	Path	When
`register`	`/neuron/register`	On SSE open
`update-capability`	`/neuron/update-capability`	On concurrency change
`heartbeat`	`/neuron/heartbeat`	Every 10s while connected
`ack`	`/neuron/ack`	Handler succeeded
`nack`	`/neuron/nack`	Handler failed
`progress`	`/neuron/progress`	During handler execution

Server → Neuron (SSE):

Frame	When
`registered`	Gateway accepted the register frame; effective capabilities returned
`lease`	Task to execute
`cancel`	Gateway cancels a specific lease (workflow cancelled)
`capability-revoked`	A capability was deauthorized mid-session
`disconnect`	Gateway is closing the connection cleanly

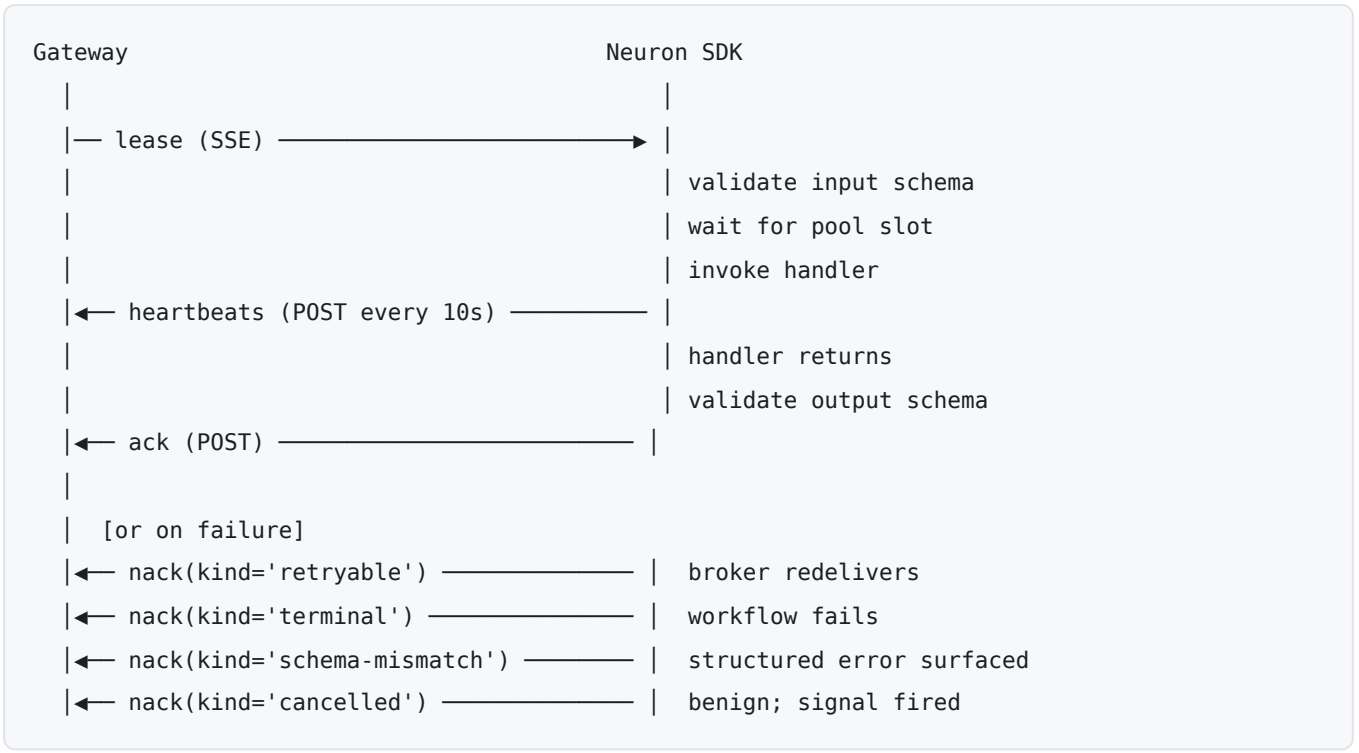
Heartbeat

Every ~10s the SDK posts:

```
{
  "type": "heartbeat",
  "neuronId": "worker-host-1234",
  "inFlight": [
    { "leaseId": "01HB...", "taskId": "task-abc", "startedAt": "2024-01-01T12:00:00Z" }
  ],
  "sentAt": "2024-01-01T12:00:10Z"
}
```

The gateway renews lease expiries for every `leaseId` in `inFlight` in a single DB round-trip. **Only running leases appear in `inFlight`** — queued leases (admitted but waiting for a concurrency slot) are excluded so the reaper doesn't mistakenly extend their TTL.

Lease lifecycle



Reconnection

The SDK uses exponential backoff (default: 500ms initial, doubling, 30s ceiling). On reconnect:

1. SSE stream re-opens.
2. `register` is posted with the full capability list.
3. Heartbeat loop resumes.

In-flight leases from before the disconnect were never acked — the broker redelivers them. The reaper flips stale `assigned` DB rows back to `pending`. Your handlers will receive the same task again; make them idempotent.

Troubleshooting

`Neuron.handle() must be called before start()`

Register all capabilities before calling `start()`. For runtime changes, use `updateCapability()`.

`Neuron.start() requires at least one handler`

You must call `neuron.handle(...)` at least once before `start()`.

Registration rejected with HTTP 403

Your JWT is not authorized for the namespace you're trying to advertise. Check:

- The capability type uses the correct namespace prefix (`core/*` requires a service identity, not a user JWT).
- For user-scoped capabilities, the `onBehalfOf` value matches the user in your JWT.

Tasks not being received

1. Check the `registered` log line — it shows the **effective** capabilities after the gateway's policy/auth filters. If a capability is missing, the gateway filtered it.
2. Check `neuronId` — if it changes across restarts, the gateway may have stale session state.
3. Check heartbeat logs — if heartbeats are failing, the lease reaper may be reclaiming tasks faster than they're running.

Handler keeps getting re-delivered

The handler completed but the `ack` POST failed (gateway returned 4xx or 5xx). Look for `ack POST rejected by gateway` in logs. Root causes:

- Output failed the `outputSchema` check — fix your handler's return shape.
- Gateway was restarting — the next delivery will succeed once it's back.

Tasks stuck in `assigned` state

Heartbeats aren't reaching the gateway, so the reaper hasn't reclaimed them. Check:

- Is the SSE stream still connected? (`transport reconnecting` log entries)
- Are heartbeat POSTs failing? (`heartbeat post failed` log entries)
- Is `heartbeatIntervalMs` misconfigured to a very large value?

Auth 401 on every request

The `auth` callback isn't returning a valid token. If using a refresh flow, ensure the callback fetches a fresh token on each call (the SDK caches nothing — it's your callback's job).

Releasing (maintainers)

`@holokai/neuron-sdk` is published to public npm. Releases are cut from a monorepo-scoped git tag, and CI does the publish — there is no npm token on anyone's laptop and none stored in the repo.

Cut a release

1. **Bump the version** in `packages/neuron-sdk/package.json` (`version`). Follow semver:
    - Pilot phase, we are in **`0.x`** . Because the protocol is still absorbing pilot-developer feedback, a **breaking** change bumps the **minor** (`0.2.0` → `0.3.0`); a backward-compatible change or fix bumps the **patch** (`0.2.0` → `0.2.1`). This is the standard `0.x` convention and is what buys us room to break things on feedback without a major bump.

- Hold **`1.0.0`** until the public API has survived pilot feedback and we're ready to commit to "breaking changes bump the major." Jumping to `1.0.0` is a stability promise, not a milestone — don't spend it early.

2. **Commit** the bump (reference the Linear ticket) and merge to `main`.
3. **Tag and push** from `main`:

   ```
   git tag neuron-sdk-v0.2.0        # must equal the package.json version
   git push origin neuron-sdk-v0.2.0
   ```

   The tag is prefixed `neuron-sdk-v` (the Python SDK uses `neuron-sdk-python-v`) so the two never collide.

4. **`publish-neuron-sdk.yml`** fires on the tag: it asserts the tag version matches `package.json`, runs the `prepublish` gate (`build` + `test`), and publishes to npm. The `npm` GitHub Environment gates the publish behind a **required-reviewer approval** — approve the run under Actions, and on success the new version is live at `@holokai/neuron-sdk`.

   > No `--provenance`: bigbrain is a private repo, so npm's provenance attestation (a public transparency-log link back to the source) adds little and has historically errored from private repos. Trusted-publishing auth is unaffected. Add the flag back if bigbrain ever goes public.

The tag→`package.json` assertion means a mismatched tag fails **before** the registry is touched — npm refuses to re-upload an existing version, so there's no half-published state to clean up.

Publish auth: npm Trusted Publishing (one-time setup)

CI authenticates to npm via **OIDC trusted publishing** — GitHub Actions exchanges a short-lived OIDC token for publish rights, so no long-lived `NPM_TOKEN` exists to leak. This mirrors the Python SDK's PyPI trusted publishing. It requires two one-time setups:

1. **On npmjs.com** (a package admin, once): open the `@holokai/neuron-sdk` package → **Settings** → **Trusted Publisher** → **GitHub Actions**, and register:
   - Organization / repository: `holok-ai/bigbrain`
   - Workflow filename: `publish-neuron-sdk.yml`
   - Environment: `npm`
2. **In this repo** (once): create the **`npm`** GitHub Environment (Settings → Environments). Add reviewers/branch protection here if you want a manual approval gate before each publish.

Until the trusted publisher is registered, the `npm publish` step fails with an auth error — that's the expected "not set up yet" signal, not a code bug.

Manual publish (fallback / first bootstrap)

The tag flow is the norm. If you ever need to publish by hand (e.g. to bootstrap the very first version before trusted publishing is configured):

```
cd packages/neuron-sdk
pnpm run publish:npm:dry   # inspect the tarball + files list
pnpm run publish:npm       # runs the prepublishOnly build+test gate, then publishes
```

This uses your own `npm login` credentials and skips the CI approval gate — prefer the tag flow for anything a developer will consume.